2301265
Data Structures Algorithm Designs and Analysis

Divide and Conquer

Assoc.Prof. Suphakant Phimoltares, Ph.D.

Advanced Virtual and Intelligent Computing (AVIC) Center

Agenda

- Concepts and design paradigm
- Merge sort
- Master theorem
- Quicksort
- Binary search
- Matrix multiplication
- Convex hull

Concepts and Design Paradigm

- Divide-and-conquer algorithms are naturally implemented as **recursive procedures**.
- Divide-and-Conquer algorithms work according to the following general plan:

1. A problem's instance is divided into several smaller instances of the same problem, ideally of about the same size.
2. The smaller instances are solved (typically recursively, however sometimes a different algorithm is used when instances becomes small).
3. If necessary, the solutions obtained for the smaller instances are combined to get a solution to the original problem.

Concepts and Design Paradigm

- For example, if we want to compute the sum of n number $a_0 + a_1 + a_2 + \cdots + a_{n-1}$
- If $n > 1$, we can divide the problem into two instances of the same problem:
  - To compute the sum of the first floor(n/2) numbers
  - To compute the sum of the remaining ceil(n/2) numbers
- Once each of these two sums is computed (by applying the same method recursively), we can add their values to get the sum in question.
- What is an efficiency for this algorithm compared to a brute force approach?

Concepts and Design Paradigm

- Not every divide and conquer algorithm is necessarily more efficiency than even a brute force solution.
- But usually time spent on executing the divide and conquer plan turns out to be smaller than solving a problem by a different method.
- It is worth to keep in mind that the divide and conquer technique is ideally suited for parallel computations, in which each sub-problem can be solved simultaneously by its own processor.

Concepts and Design Paradigm

- Divide-and conquer is a general algorithm design paradigm:

1. **Divide** the problem into a number of subproblems that are smaller instances of the same problem.
2. **Conquer** the subproblems by solving them recursively.

   if the subproblem sizes are small enough, solve them by brute force.
3. **Combine** the solutions to the subproblems into the solution for the original problem.

- Analysis can be done using recurrence equations.

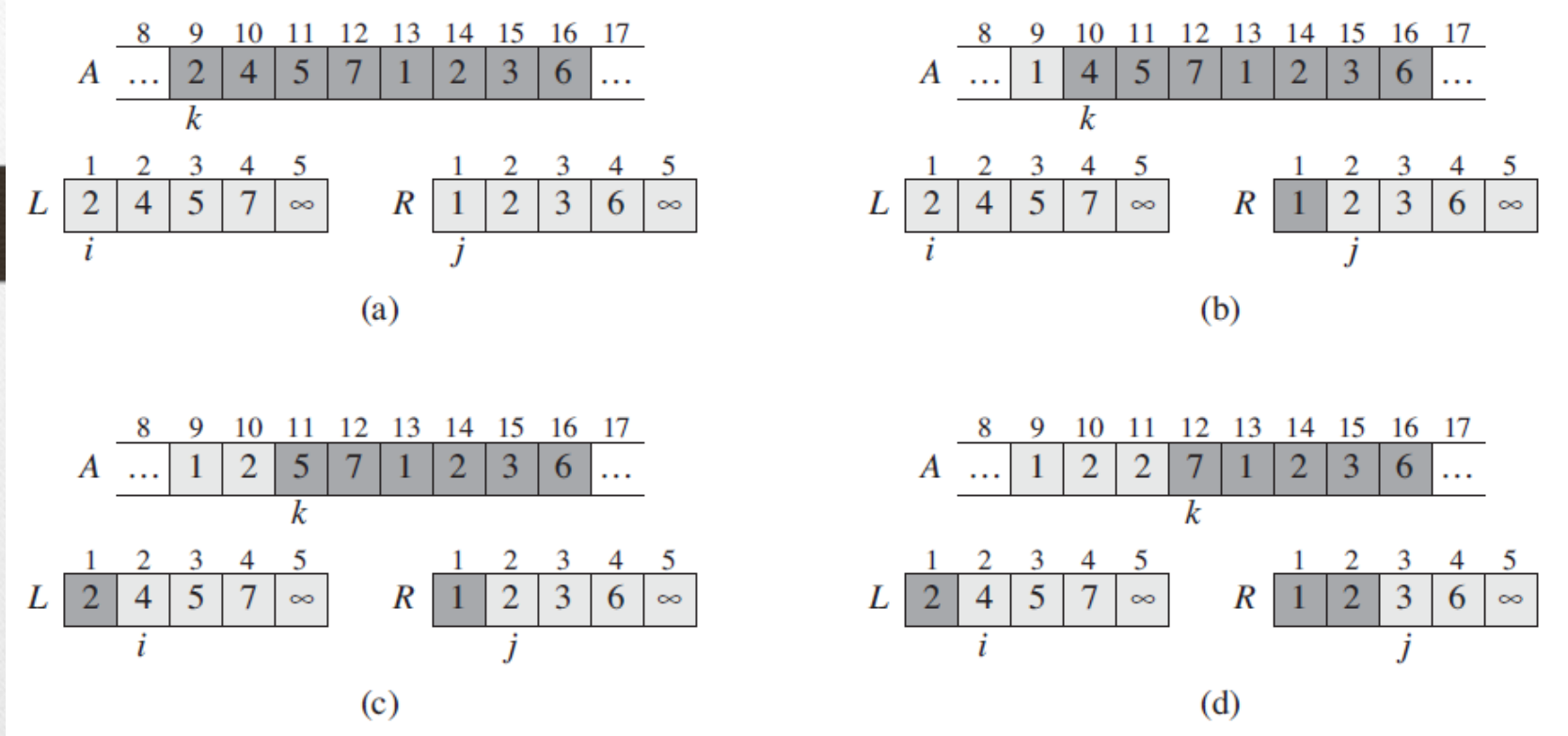
Merge Sort

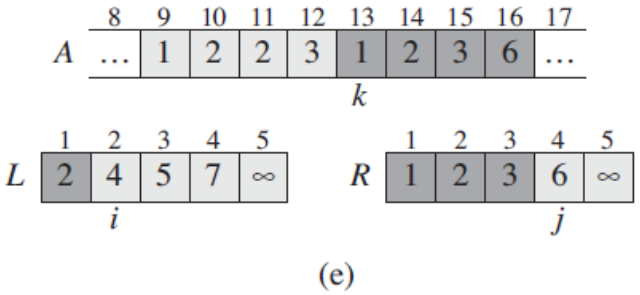
- **Divide**: Divide the n-element sequence to be sorted into two subsequences of $n/2$ elements each.
- **Conquer:** Sort the two subsequences recursively using merge sort.
- **Combine:** Merge the two sorted subsequences to produce the sorted answer.

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

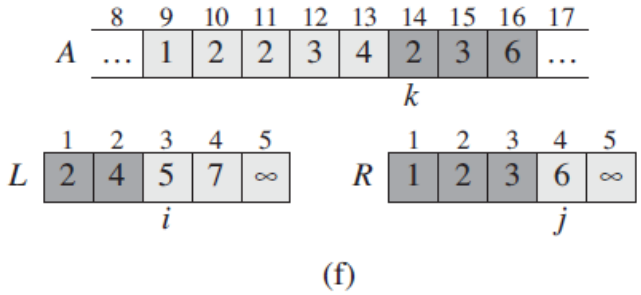
```
MERGE(A, p, q, r)
 1  n1 = q − p + 1
 2  n2 = r − q
 3  let L[1 .. n1 + 1] and R[1 .. n2 + 1] be new arrays
 4  for i = 1 to n1
 5      L[i] = A[p + i − 1]
 6  for j = 1 to n2
 7      R[j] = A[q + j]
 8  L[n1 + 1] = ∞
 9  R[n2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
```

Merge Procedure

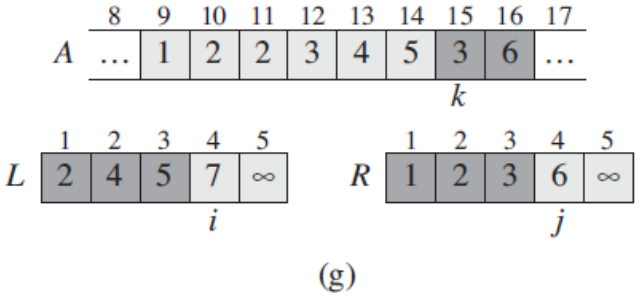



8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 1 2 3 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

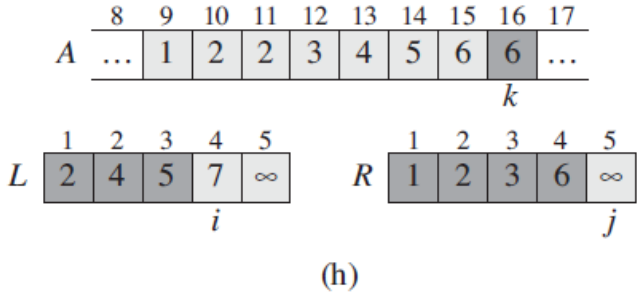
(e)


8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 2 3 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

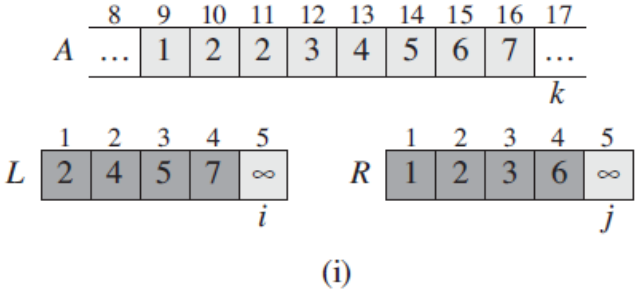
(f)


8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 5 3 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

(g)

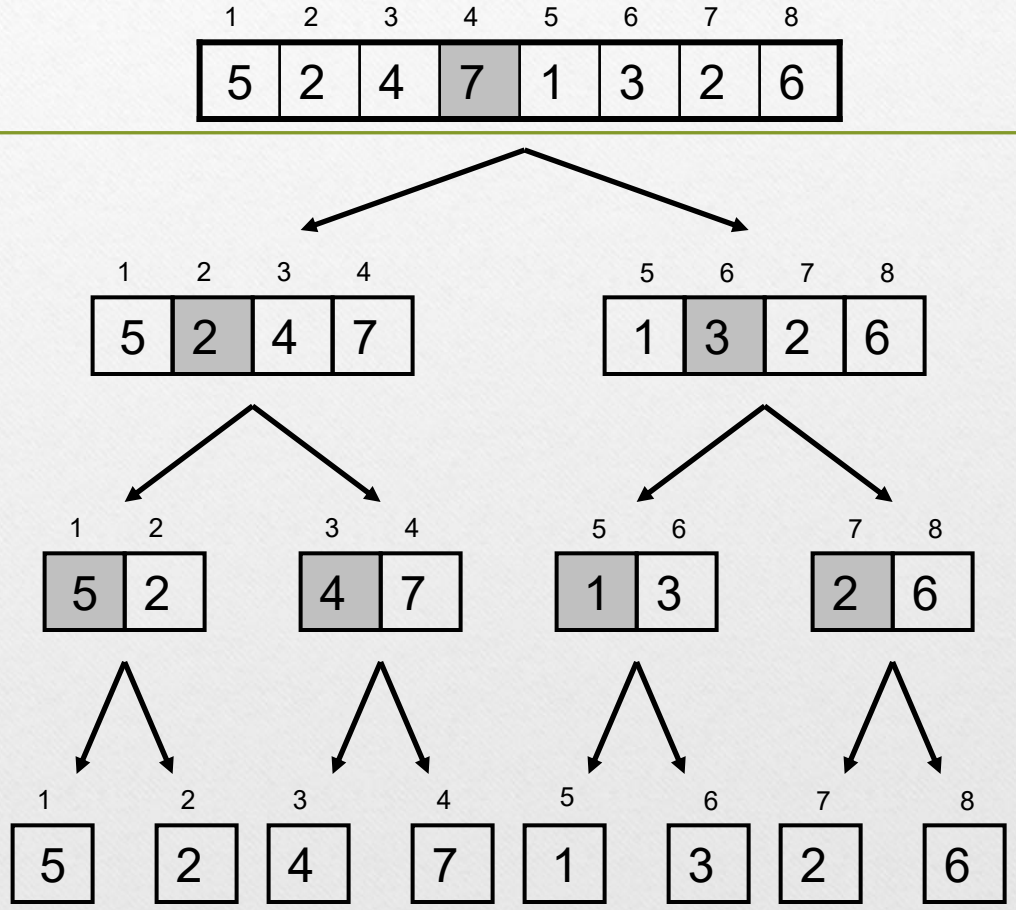

8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 5 6 6 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

(h)

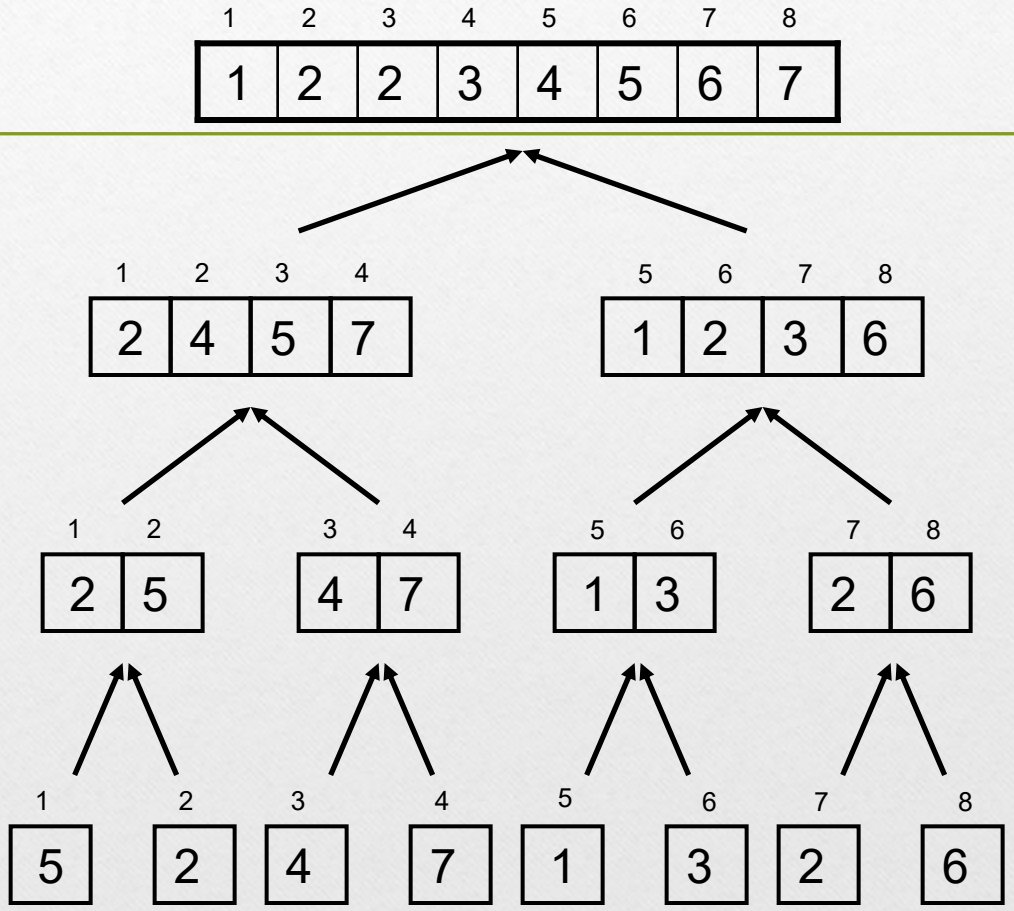

8 9 10 11 12 13 14 15 16 17
A … 1 2 2 3 4 5 6 7 …
k
1 2 3 4 5
L 2 4 5 7 ∞
i
1 2 3 4 5
R 1 2 3 6 ∞
j

(i)

Example


1 2 3 4 5 6 7 8
5 2 4 7 1 3 2 6
1 2 3 4
5 2 4 7
5 6 7 8
1 3 2 6
1 2
5 2
3 4
4 7
5 6
1 3
7 8
2 6
1
5
2
3
4
7
5
1
6
3
7
2
8
6

Example


1 2 3 4 5 6 7 8
1 2 2 3 4 5 6 7
1 2 3 4
2 4 5 7
5 6 7 8
1 2 3 6
1 2
2 5
3 4
4 7
5 6
1 3
7 8
2 6
1
5
2
3
4
7
5
1
6
3
7
2
8
6

Analysis of Merge Sort

- **Divide:** The divide step just computes the middle of the subarray, which takes constant time $\Theta(1)$.
- **Conquer:** We recursively solve two subproblems, each of size $n/2$, which contributes $2T(n/2)$ to the running time.
- **Combine:** the MERGE procedure on an $n-$element subarray takes time $\Theta(n)$.
- The worst-case running time $T(n)$ of merge sort:

$$T(n) = \begin{cases} \Theta(1) & if\ n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & if\ n > 1 \end{cases}$$

Running Time: $T(n) =?$

Recurrences

- A ***recurrence*** is an equation or inequality that describes a function in terms of its value on smaller inputs.
- Examples

$$T(n) = \begin{cases} 1 & if\ n = 1 \\ T(n-1) + 1 & if\ n > 1 \end{cases}$$ Solution: $T(n) = n$

$$T(n) = \begin{cases} 1 & if\ n = 1 \\ 2T\left(\frac{n}{2}\right) + n & if\ n > 1 \end{cases}$$ Solution: $T(n) = nlogn + n$

Solving Recurrences

- Iterative Substitution
- Recursion Tree
- Master Method

Iterative Substitution

- $T(n) = \begin{cases} c & if\ n = 1 \\ 2T(n/2) + cn & if\ n > 1 \end{cases}$
- $T(n) = 2T\left(\frac{n}{2}\right) + cn = 2\left(\left(2T\left(\frac{n}{2^2}\right)\right) + c\left(\frac{n}{2}\right)\right) + cn$

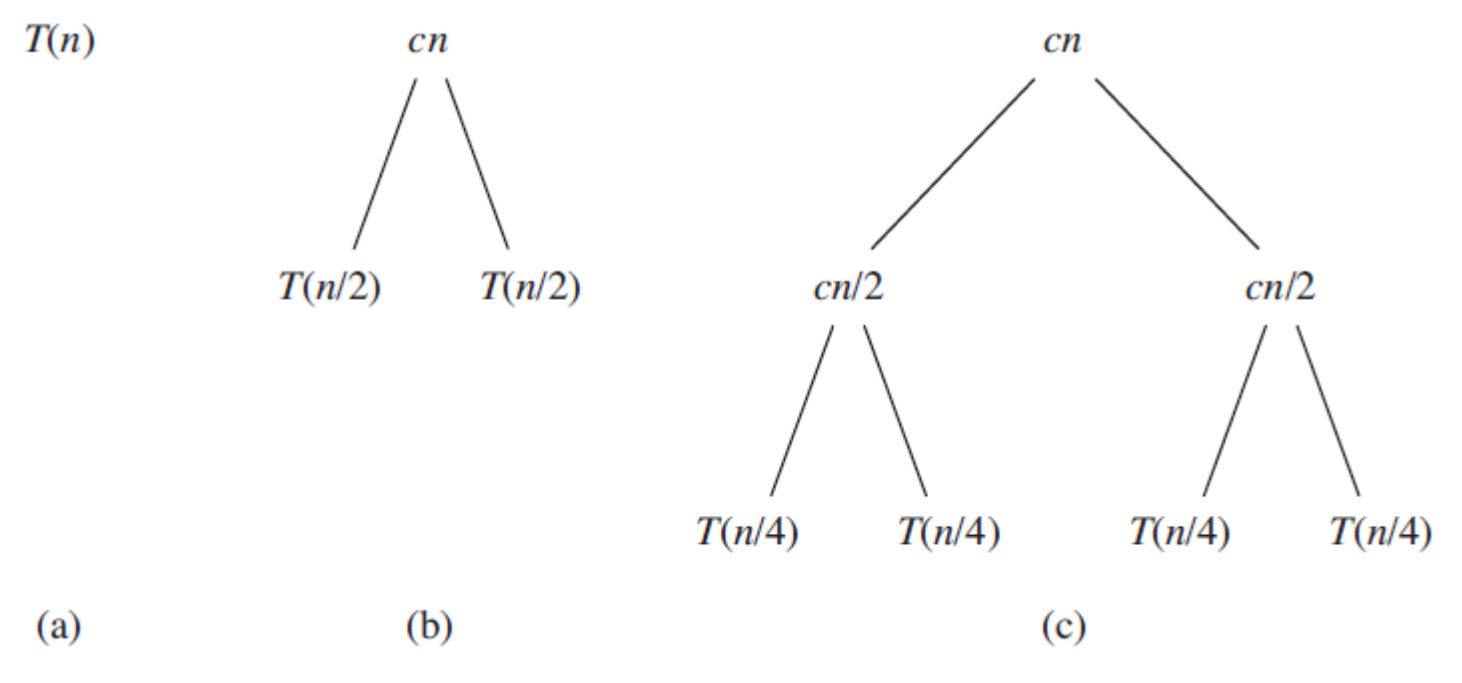
$= 2^2T\left(\frac{n}{2^2}\right) + 2cn$

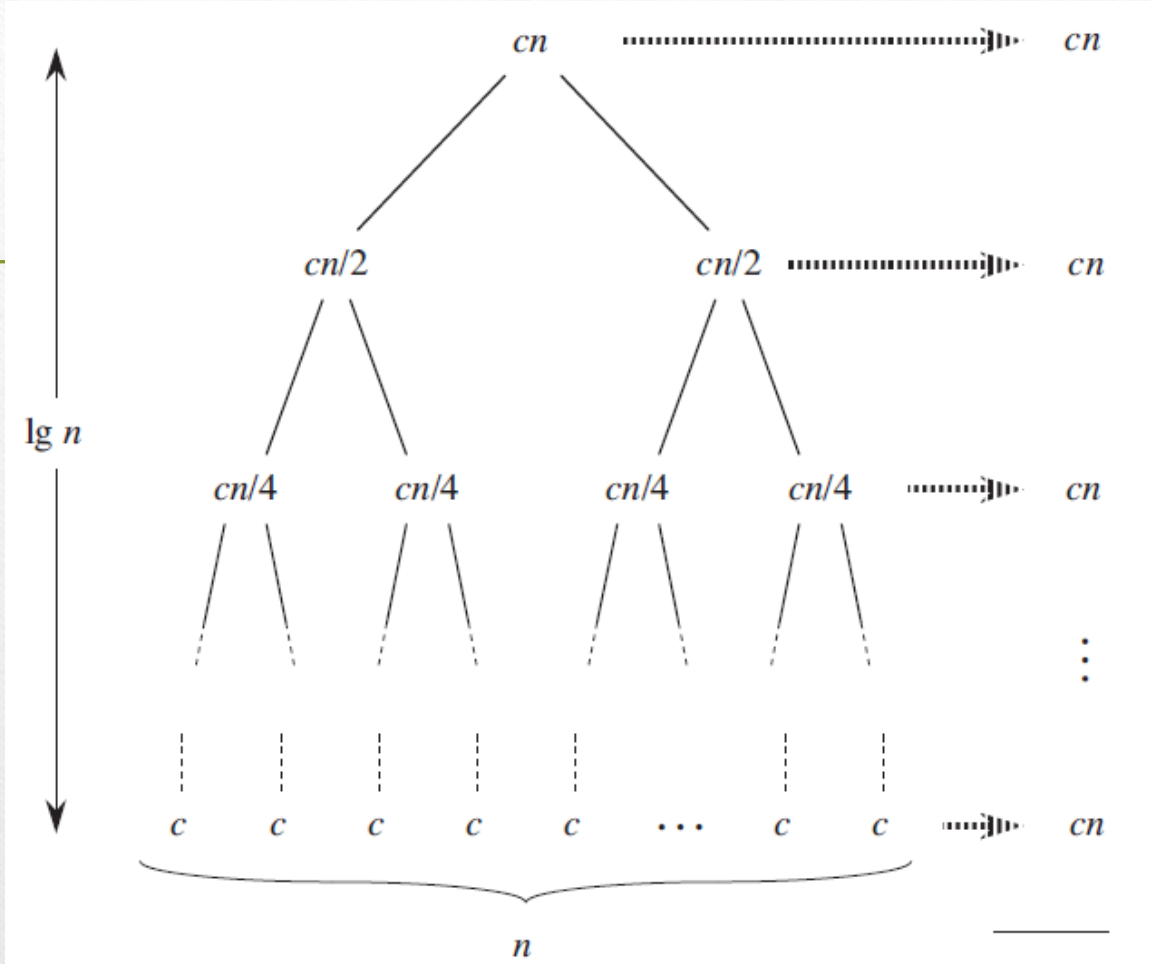
$= 2^3T\left(\frac{n}{2^3}\right) + 3cn$

$= \cdots = 2^iT\left(\frac{n}{2^i}\right) + icn$

- Note that base, $T(n) = c$, case occurs when $2^i = n$. That is, $i = \log n$.
- So, $T(n) = cn + cnlogn$
- Thus, $T(n)$ is $O(n \log n)$.

Recursion Tree




cn
cn
cn/2
cn/2
cn
lg n
cn/4
cn/4
cn/4
cn/4
cn
c
c
c
c
c
c
c
cn
n
(d)
Total: cn lg n + cn

The Master Theorem

- Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence $T(n) = aT(n/b) + f(n)$

  where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$.

  Then $T(n)$ has the following asymptotic bounds:

The Master Theorem

1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a}(\log n)^k)$, then $T(n) = \Theta(n^{\log_b a}(\log n)^{k+1})$.
3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficient large n, then $T(n) = \Theta(f(n))$.

Master Method

- Ex.1 $T(n) = 4T\left(\frac{n}{2}\right) + n$
- Ex.2 $T(n) = 2T\left(\frac{n}{2}\right) + n\log n$
- Ex.3 $T(n) = T\left(\frac{n}{3}\right) + n\log n$
- Ex.4 $T(n) = 8T\left(\frac{n}{2}\right) + n^2$
- Ex.5 $T(n) = 9T\left(\frac{n}{3}\right) + n$

Master Method

- Ex.6 $T(n) = T\left(\frac{n}{2}\right) + 1$
- Ex.7 $T(n) = 2T\left(\frac{n}{2}\right) + \log n$
- Ex.8 $T(n) = T(2n/3) + 1$
- Ex.9 $T(n) = 3T\left(\frac{n}{4}\right) + n \log n$

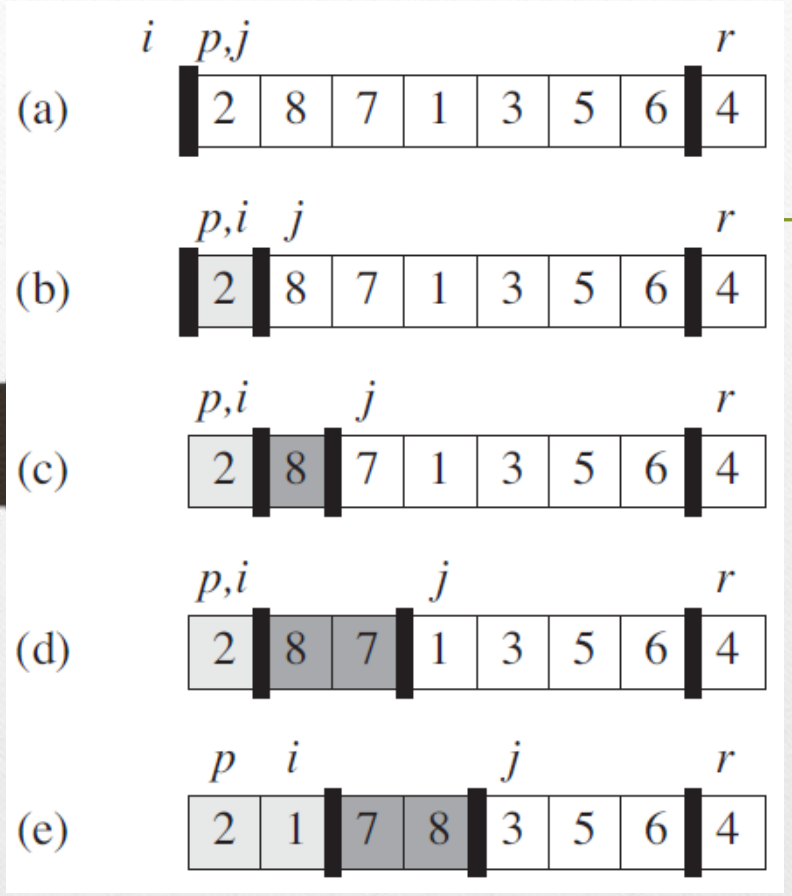
Quicksort

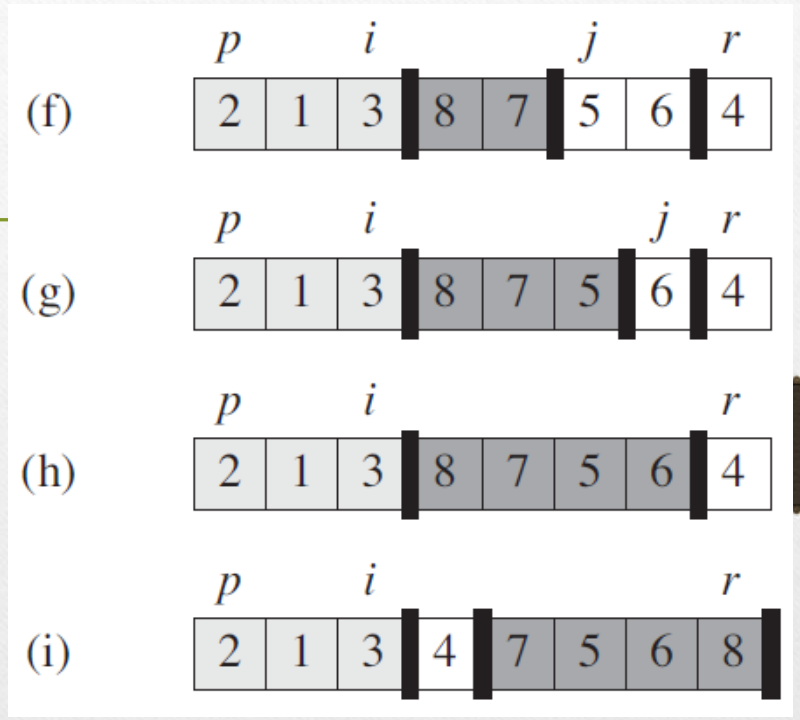
- **Divide:** Partition (rearrange) the array $A[p..r]$ into two (possibly empty) subarrays $A[p..q-1]$ and $A[q+1..r]$ such that each element of $A[p..q-1]$ is less than or equal to $A[q]$, which is, in turn, less than or equal to each element of $A[q+1..r]$. Compute the index q as part of this partitioning procedure.
- **Conquer:** Sort the two subarrays $A[p..q-1]$ and $A[q+1..r]$ by recursive calls to quicksort.
- **Combine:** Because the subarrays are already sorted, no work is needed to combine them: the entire array $A[p..r]$ is now sorted.

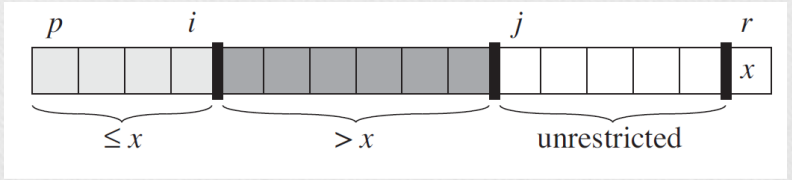
```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Running time: $\Theta(n)$ to partition an n-element subarray


i p,j r
(a) 2 8 7 1 3 5 6 4
p,i j r
(b) 2 8 7 1 3 5 6 4
p,i j r
(c) 2 8 7 1 3 5 6 4
p,i j r
(d) 2 8 7 1 3 5 6 4
p i j r
(e) 2 1 7 8 3 5 6 4


p i j r
(f) 2 1 3 8 7 5 6 4
p i j r
(g) 2 1 3 8 7 5 6 4
p i r
(h) 2 1 3 8 7 5 6 4
p i r
(i) 2 1 3 4 7 5 6 8


p i j r x
≤ x
> x
unrestricted

Performance of Quicksort

- The running time of quicksort depends on whether the partitioning is balanced or unbalanced.
- If the partitioning is balanced, the algorithm runs asymptotically as fast as merge sort.
- If the partitioning is unbalanced, however, it can run asymptotically as slowly as insertion sort.

Analysis of Quicksort

- Worst-case partitioning
  - The worst-case behavior for quicksort occurs when the partitioning routine produces one subproblem with $n - 1$ elements and one with 0 elements.
  - The recurrence for the running time is

$$T(n) = T(n-1) + T(0) + \Theta(n)$$
$$= T(n-1) + \Theta(n)$$

  - This recurrence has the solution $T(n) = \Theta(n^2)$.

Analysis of Quicksort

- Best-case partitioning
  - occurs when the subarrays are completely balanced every time.
  - The recurrence for the running time is

$$T(n) = 2T(n/2) + \Theta(n)$$

  - By case 2 of the master theorem, this recurrence has the solution $T(n) = \Theta(n \log n)$

Analysis of Quicksort

- Average-case partitioning
- The average-case running time of quicksort is much closer to the best case than to the worst case.
- The recurrence for the running time is

$$T(n) = \frac{1}{n}\sum_{k=1}^{n}\big(T(k-1) + T(n-k)\big) + (n-1)$$

- This recurrence has the solution $T(n) = \Theta(nlogn)$

Randomized Quicksort

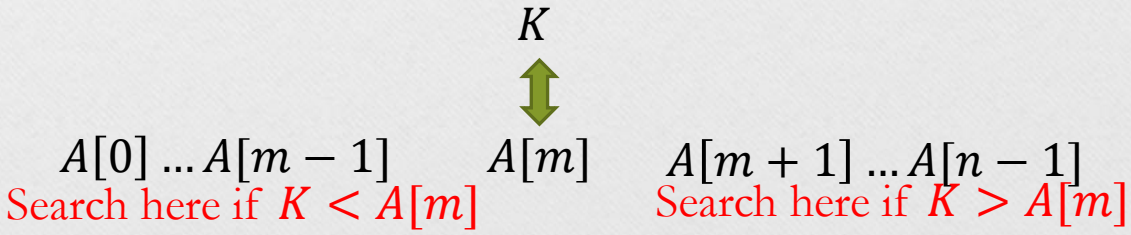
- Instead of always using $A[r]$ as the pivot, a randomly chosen element is selected from the subarray $A[p..r]$.
- Then, the element $A[r]$ is exchanged with an element chosen at random from $A[p..r]$.

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r
2      q = RANDOMIZED-PARTITION(A, p, r)
3      RANDOMIZED-QUICKSORT(A, p, q − 1)
4      RANDOMIZED-QUICKSORT(A, q + 1, r)
```

Binary Search

- If the sequence A is sorted, we can check the midpoint of the sequence against K and eliminate half of the sequence from further consideration. The binary search algorithm repeats this procedure, halving the size of the remaining portion of the sequence each time.



Binary Search

Ex. How many comparisons does the algorithm make on an array of n elements?

Ex. Use the master method to show that the solution to the binary-search recurrence $T(n) = T(n/2) + \Theta(1)$

Matrix Multiplication

- If $A = [a_{ij}]$ and $B = [b_{ij}]$ are square $n \times n$ matrices, then in the product $C = A \times B$, we define the entry c_{ij}, for $i, j = 1, 2, \ldots, n$, by

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

Matrix Multiplication

- Method 1: Brute Force

```
SQUARE-MATRIX-MULTIPLY(A, B)
1  n = A.rows
2  let C be a new n × n matrix
3  for i = 1 to n
4      for j = 1 to n
5          c_ij = 0
6          for k = 1 to n
7              c_ij = c_ij + a_ik · b_kj
8  return C
```

- Analysis: three nested loops, each iterates n times, and the innermost loop body takes constant time $\Rightarrow \Theta(n^3)$

Matrix Multiplication

- Method 2: Simple Divide and Conquer
- Assume that n is an exact power of 2 in each of the $n \times n$ matrices.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

- Partition each of $A, B,$ and C into four $n/2 \times n/2$ matrices.

Matrix Multiplication

- Rewrite the equation $C = A \times B$ as

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

- Thus, there are four equations
  - $C_{11} = A_{11}B_{11} + A_{12}B_{21}$
  - $C_{12} = A_{11}B_{12} + A_{12}B_{22}$
  - $C_{21} = A_{21}B_{11} + A_{22}B_{21}$
  - $C_{22} = A_{21}B_{12} + A_{22}B_{22}$

```
SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)
 1  n = A.rows
 2  let C be a new n × n matrix
 3  if n == 1
 4       c11 = a11 · b11
 5  else partition A, B, and C as in equations (4.9)
 6       C11 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B11)
                + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B21)
 7       C12 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A11, B12)
                + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A12, B22)
 8       C21 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B11)
                + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B21)
 9       C22 = SQUARE-MATRIX-MULTIPLY-RECURSIVE(A21, B12)
                + SQUARE-MATRIX-MULTIPLY-RECURSIVE(A22, B22)
10  return C
```

Analysis of Simple Divide and Conquer

- Let $T(n)$ be the time to multiply two $n \times n$ matrices
- Base case: $n = 1 \Rightarrow T(n) = \Theta(1)$
- Recursive case: $n > 1$
  - **Divide**: take $\Theta(1)$ time using index calculation; otherwise, $\Theta(n^2)$ time
  - **Conquer**: make 8 recursive calls, each multiplying $n/2 \times n/2$ matrices $\Rightarrow 8T(n/2)$
  - **Combine**: take $\Theta(n^2)$ time to add $n/2 \times n/2$ matrices four times

Analysis of Simple Divide and Conquer

- Recurrence

$$T(n) = \begin{cases} \Theta(1) & if\ n = 1 \\ 8T(n/2) + \Theta(n^2) & if\ n > 1 \end{cases}$$

- The solution is $T(n) = ?$

Matrix Multiplication

- Method 3: Strassen's Method
- Instead of performing eight recursive multiplications of $n/2 \times n/2$ matrices, it performs only seven.

Strassen's Method

1. Divide the input matrices A and B and output matrix C into $n/2 \times n/2$ submatrices. This step takes $\Theta(1)$ time by index calculation.
2. Create 10 matrices $S_1, S_2, \ldots, S_{10}$, each of which is $n/2 \times n/2$ and is the sum or difference of two matrices created in step 1. This step takes $\Theta(n^2)$ time.
3. Using the submatrices created in step 1 and the 10 matrices created in step 2, recursively compute seven matrix products P_1, $P_2, \ldots, P_7$. Each matrix P_i is $n/2 \times n/2$. This step takes $7T(n/2)$ time.
4. Compute the desired submatrices $C_{11}, C_{12}, C_{21}, C_{22}$ of the result matrix C by adding and subtracting various combinations of the P_i matrices. This step takes $\Theta(n^2)$ time.

Strassen's Method

- The running time $T(n)$ of Strassen's algorithm:

$$T(n) = \begin{cases} \Theta(1) & if\ n = 1 \\ 7T(n/2) + \Theta(n^2) & if\ n > 1 \end{cases}$$

- The solution is $T(n)$ =?

Strassen's Method

- Step 2: create 10 matrices

$$S_1 = B_{12} - B_{22} \quad S_2 = A_{11} + A_{12} \quad S_3 = A_{21} + A_{22}$$
$$S_4 = B_{21} - B_{11} \quad S_5 = A_{11} + A_{22} \quad S_6 = B_{11} + B_{22}$$
$$S_7 = A_{12} - A_{22} \quad S_8 = B_{21} + B_{22} \quad S_9 = A_{11} - A_{21}$$
$$S_{10} = B_{11} + B_{12}$$

  - add or subtract $n/2 \times n/2$ matrices 10 times $\Rightarrow$ take $\Theta(n^2)$ time.

Strassen's Method

- Step 3: recursively multiply $n/2 \times n/2$ matrices seven times to compute the following $n/2 \times n/2$ matrices:

$$P_1 = A_{11} \times S_1 \quad P_2 = S_2 \times B_{22}$$

$$P_3 = S_3 \times B_{11} \quad P_4 = A_{22} \times S_4$$

$$P_5 = S_5 \times S_6 \quad P_6 = S_7 \times S_8$$

$$P_7 = S_9 \times S_{10}$$

- recursively multiply $n/2 \times n/2$ matrices 7 times $\Rightarrow$ takes $7T(n/2)$ time.

Strassen's Method

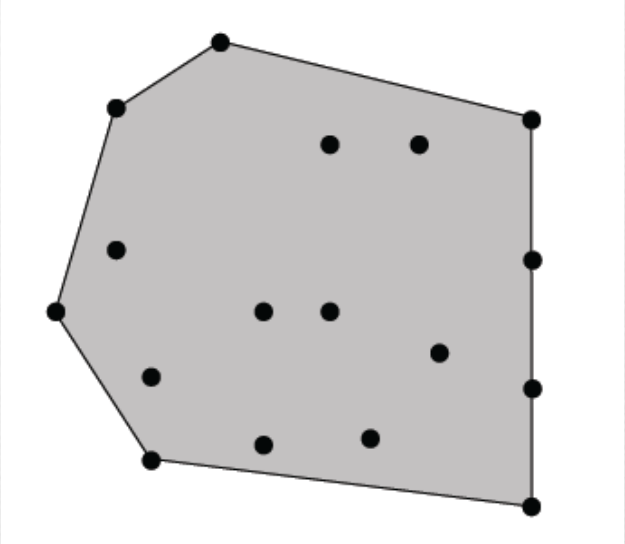
- Step 4: adds and subtracts the P_i matrices created in step 3 to construct the four $n/2 \times n/2$ submatrices of the product C.

$$C_{11} = P_5 + P_4 - P_2 + P_6$$
$$C_{12} = P_1 + P_2$$
$$C_{21} = P_3 + P_4$$
$$C_{22} = P_5 + P_1 - P_3 - P_7$$

- add or subtract $n/2 \times n/2$ matrices eight times $\Rightarrow$ takes $\Theta(n^2)$ time.

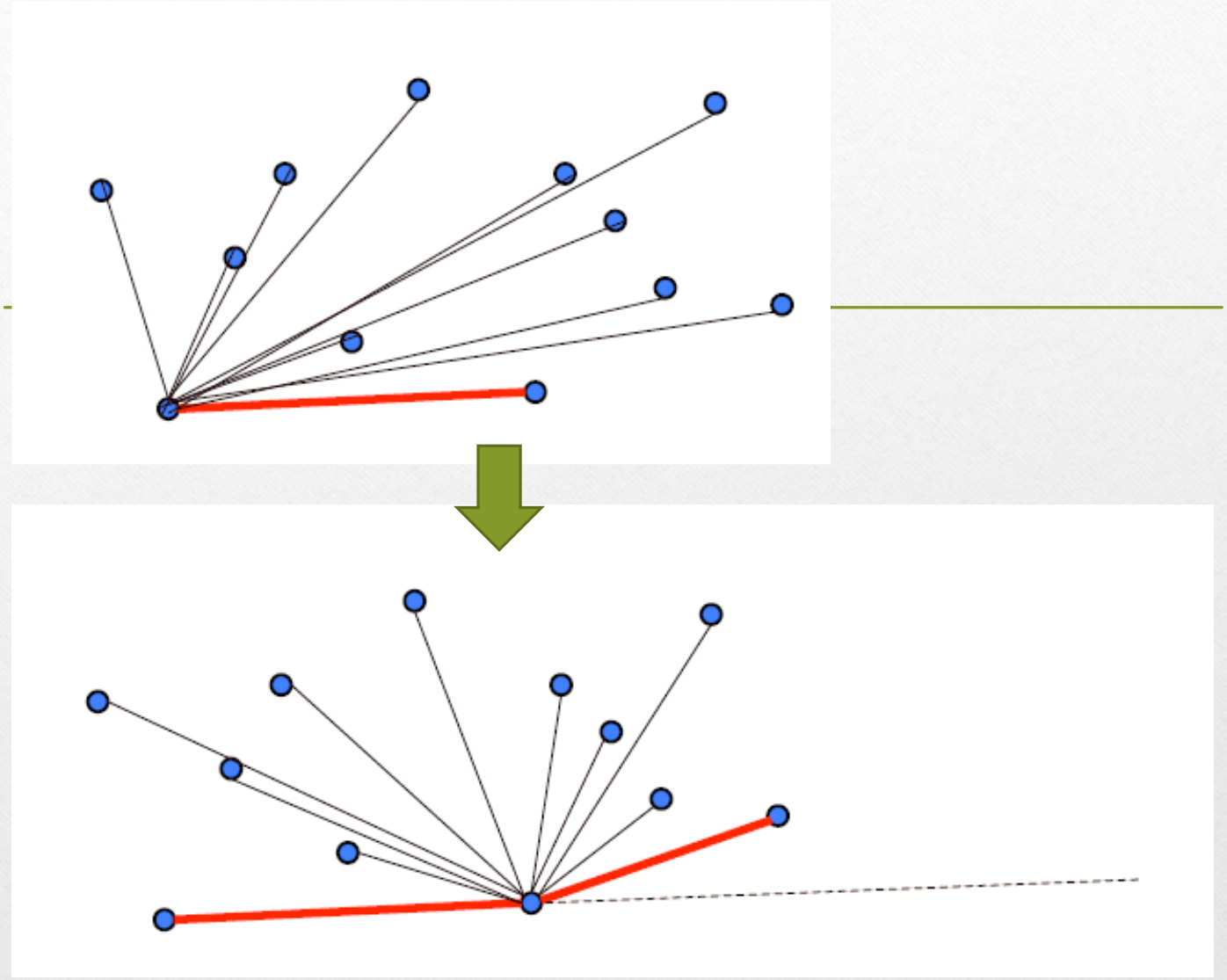
Convex Hull

- The convex hull of a set of N points is the smallest perimeter fence enclosing the points.



Convex Hull

- Method 1: Gift Wrapping
- Starting at the bottom point (bottom left if there are ties), "wrap" around the convex hull by repeatedly identifying the point with smallest polar angle relative to previous segment and add to hull – All points identified in the process are the vertices of the convex hull.

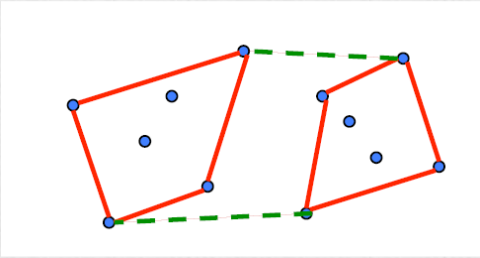


Convex Hull

- Analysis: We have to consider…
  - Time to find the bottom-most vertex.
  - Time to compute the minimum polar angle with respect to other points.
  - $T(n) = ?$

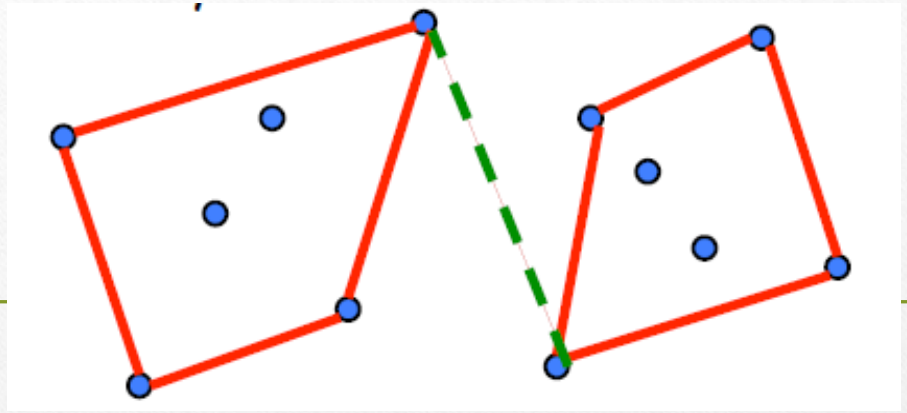
Convex Hull

- Method 2: Divide and Conquer
- Overall Goal: find upper tangent and lower tangent lines that connect the two convex hulls.
  - Upper tangent is defined as line connecting the two convex hulls such that all other hull points lie below it.
  - Lower tangent defined oppositely.

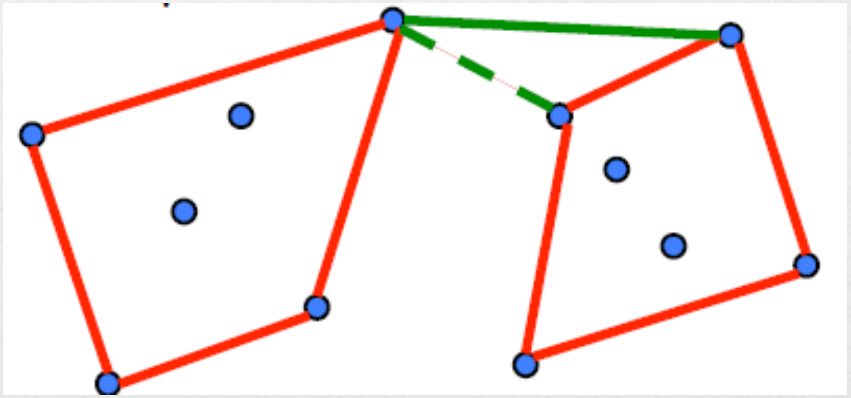


Convex Hull

- Merging Convex Hulls: Finding the Upper Tangent
  - Start at the rightmost vertex of CH1 and the leftmost vertex of CH2.
  - Upper tangent is a line segment such that it makes a left turn with next counter-clockwise vertex of CH1 and makes a right turn with next clockwise vertex of CH2.
- If the line segment connecting them is not the upper tangent
  - If the line segment does not make a left turn with next counterclockwise vertex of CH1, rotate to the next counter-clockwise vertex in CH1.
  - Else if the line segment does not make a right turn with the next clockwise vertex of CH2, rotate to the next clockwise vertex in CH2.
  - Repeat as necessary.

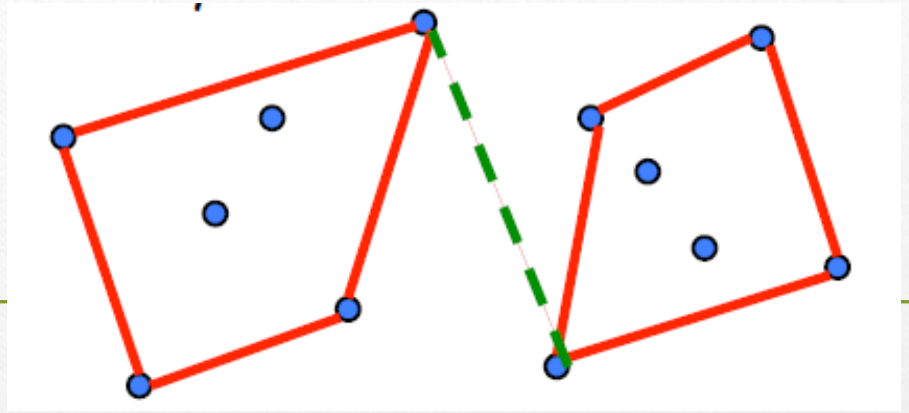




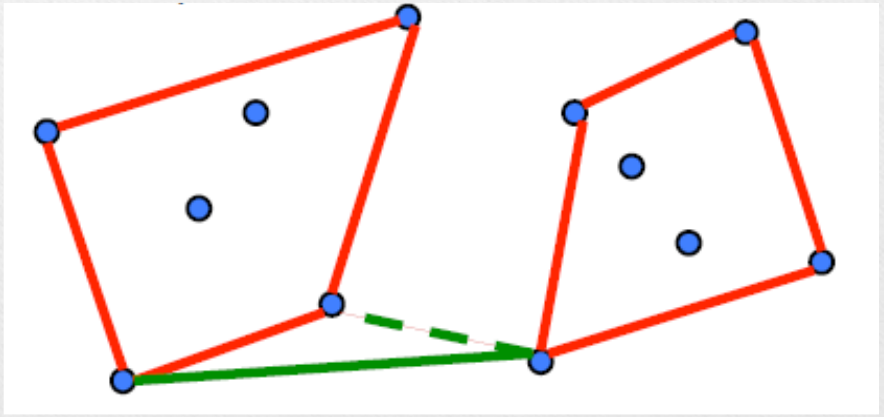


Convex Hull

- Merging Convex Hulls: Finding the Lower Tangent
  - Start at the rightmost vertex of CH1 and the leftmost vertex of CH2.
  - Lower tangent will be a line segment such that it makes a right turn with next clockwise vertex of CH1 and makes a left turn with next counterclockwise vertex of CH2.
- If the line segment connecting them is not the lower tangent
  - If the line segment does not make a right turn with next clockwise vertex of CH1, rotate to the next clockwise vertex in CH1.
  - Else if the line segment does not make a left turn with the next counter-clockwise vertex of CH2, rotate to the next counterclockwise vertex in CH2.
  - Repeat as necessary.







Convex Hull

- Analysis: We have to consider…
  - Time to sort the points: $T_S(n) = ?$
  - Time for recursive steps: $T_R(n) = ?$
    - Compute the convex hull of the left $\frac{1}{2}$ of the points.
    - Compute the convex hull of the right $\frac{1}{2}$ of the points.
    - Merge the hulls into a single convex hull.
  - $T(n) = T_S(n) + T_R(n) = ?$

Acknowledgements & References

1. T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction to Algorithms, 3rd edition, The MIT Press, 2009.
2. M.T. Goodrich and R. Tamassia, Algorithm Design and Applications, Wiley, 2015.
3. R. Sedgewick and K. Wayne, Algorithms, 4th edition, Addison-Wesley, 2011.